APPLIED MACHINE LEARNING | OPENCAMPUS.SH

Rowing Technique Analysis

Stroke-by-stroke quality evaluation

Clara Brilke 25 June 2026

THE GOAL

Aim

Train a model that evaluates stroke quality on a rowing ergometer — automatically, after every single stroke.

Per-stroke

Feedback at the granularity of one rowing stroke

Video-only

No wearables or on-body sensors required

Real-time-capable

Designed to score strokes as they happen

Scope so far

Binary task — good stroke vs. failed stroke. A working pipeline, normalization, a leak-free split and first baselines are already in place.

OUTLINE

What this talk covers

1. **Motivation**
   Why automate stroke assessment
2. **Related work**
   Sensor- and pose-based approaches
3. **Approach & pipeline**
   MediaPipe → CNN → classification
4. **Data preparation**
   Landmarks, downsampling, normalization
5. **Segmentation & split**
   Per-stroke handling, leak-free split
6. **Biggest Fail: The imbalance problem**
   355 good vs 21 bad strokes
7. **Baseline & autoencoder**
   Random Forest and LSTM-AE
8. **Next steps**
   Towards richer feedback

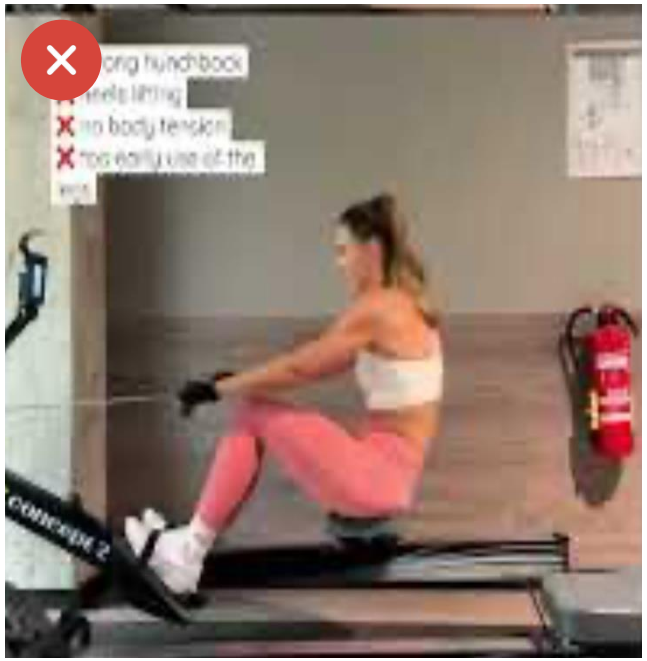
WHY IT MATTERS

Motivation

Manual technique assessment relies on experienced coaches. It is time-consuming, subjective and does not scale.

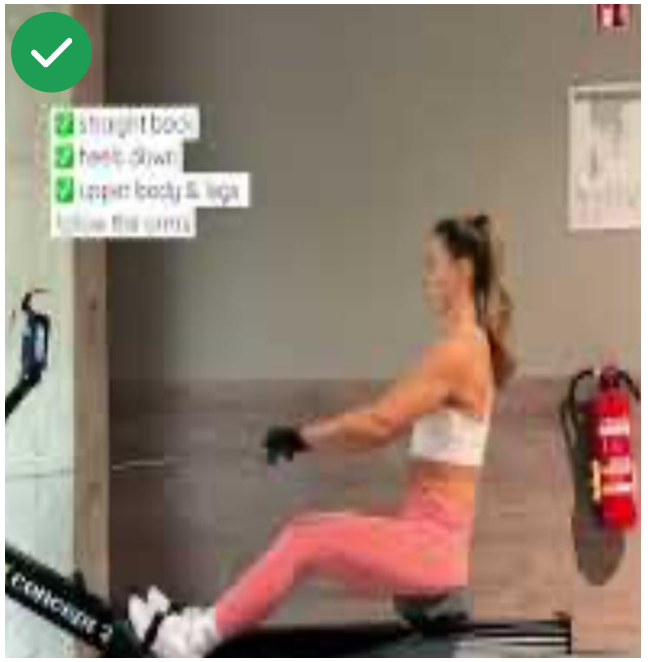
The goal

Automatically classify rowing strokes directly from ergometer video — giving every athlete instant feedback.



Faulty technique

Hunched back · heels lifting · early arm pull



Good technique

Straight back · heels down · legs then arms

Wearable inertial sensors

Bosch et al. (2015) — *Analysis of indoor rowing motion using wearable inertial sensors*

Objective
Distinguish experienced from novice rowers.

Method
k-Nearest-Neighbours

Findings
Clear differences in stroke-timing consistency — but no difference in absolute posture angles.

Limitation
Relies on on-body sensors — the athlete has to wear hardware on every session.

My approach avoids this — a camera is all that is needed.

Pose-based deep learning

Garg et al. (2022) — *Yoga pose classification: a CNN and MediaPipe inspired deep-learning approach*

Objective

Classify yoga postures; compare models with vs. without skeletonization.

Method

MediaPipe skeletonization, then CNN and transfer-learning models.

Outcome

Skeletonization clearly boosts performance — and a custom CNN benefits most, beating the transfer-learning models.

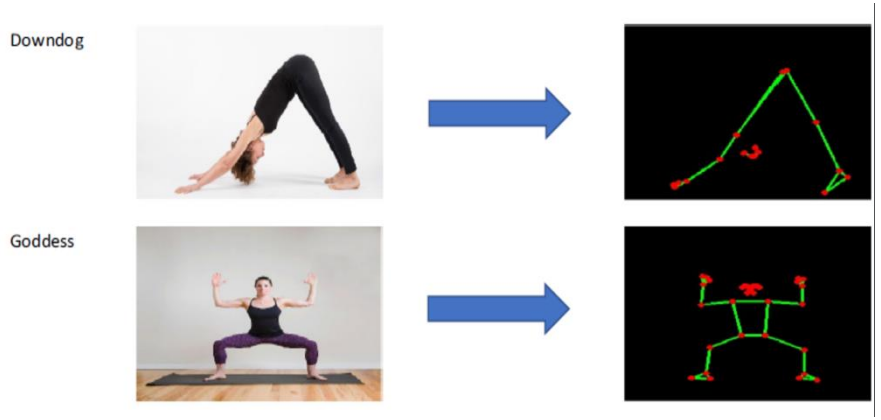


Image → skeleton → classification (Garg et al.)

Takeaway Skeletonization + a custom CNN is the most promising route — this directly motivates my design.

METHOD

My approach

1

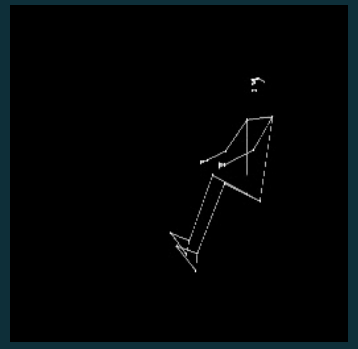
Pose extraction
MediaPipe landmarks from ergometer video

2

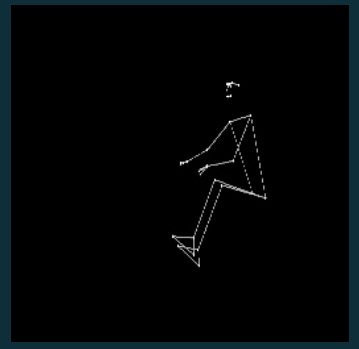
Custom CNN
Trained on the landmark sequences

3

Binary classification
Good stroke vs. bad stroke



Good Stroke



Bad Stroke

DATA · 1

Data preparation

1

Landmark extraction

MediaPipe coordinates stored as CSV — 69 features per frame.

2

Downsampling

30 fps → 10 fps: removes redundancy while keeping the motion.

3

Normalization

Makes videos from different angles and body proportions comparable.

Landmark data · CSV

	# nose_x_norm	# nose_y_norm	# left_eye_inner_x_norm	# left_eye_inner_y_norm	# left_eye_x_norm	# lef
0	-0.3688188654256076	-1.499531322894042	-0.3214116674563315	-1.5645224343482689	-0.3024613316701136	
1	-0.4166256393291495	-1.5121898688925148	-0.3786198677715513	-1.5732299926178104	-0.3650411456792083	
2	-0.4722521951124138	-1.545028285760036	-0.4288436963455196	-1.5989015214528304	-0.4154621898430503	
3	-0.4964580201333761	-1.5595082585992703	-0.449763248170749	-1.6146678054158308	-0.4376289771565364	
4	-0.4832066024237302	-1.5605791025040376	-0.4358787806670243	-1.6186007252568697	-0.4223794347872516	

67
features / frame

30→10
fps reduction

CSV
per-frame export

MediaPipe returns a pose for every frame — 0 frames without a detected pose across the dataset.

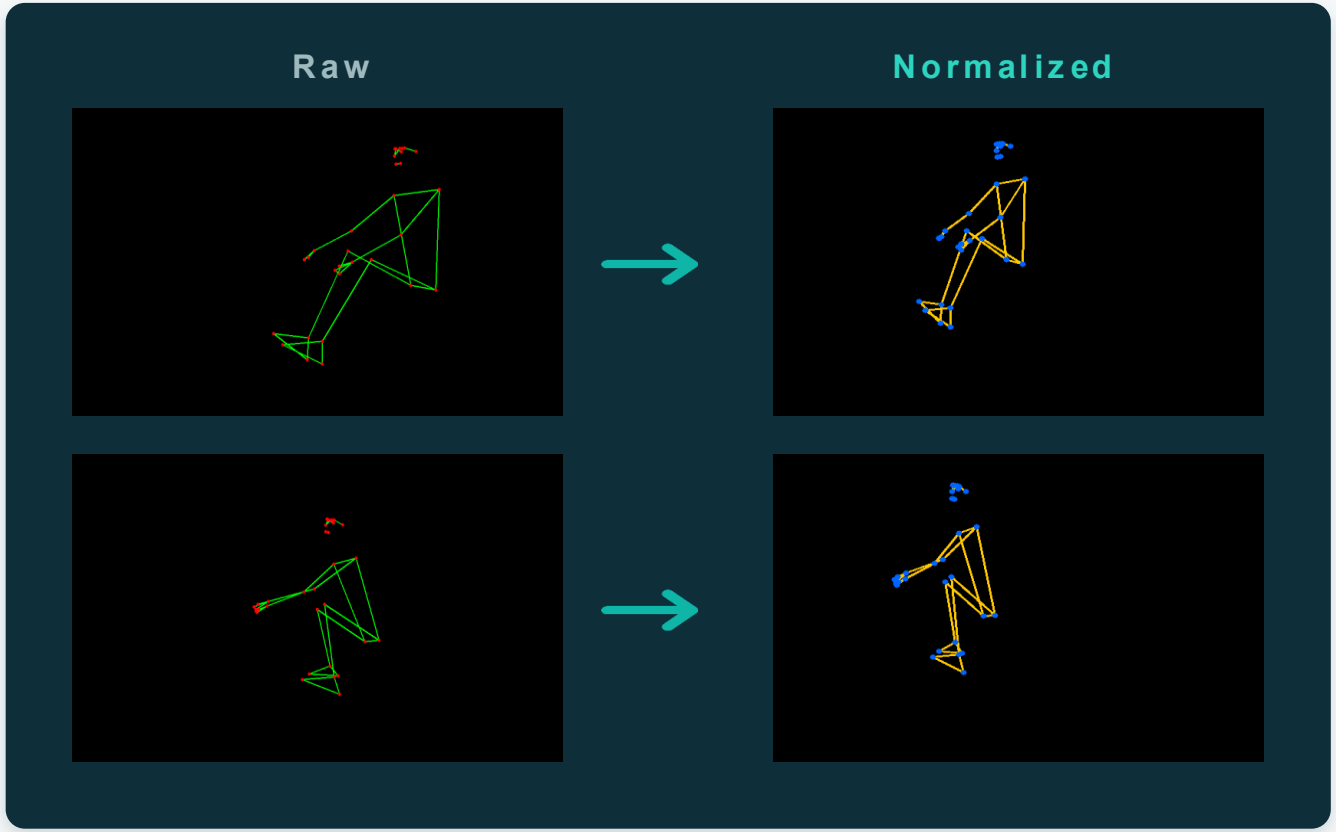
DATA · 2

Normalization

Why. Recordings differ in camera angle and in the rower's body proportions, so raw coordinates are not comparable across videos.

How. Centre each pose on a hip reference and scale by torso length — making strokes directly comparable.

↔ Same stroke, same representation — regardless of who is rowing or where the camera stands.



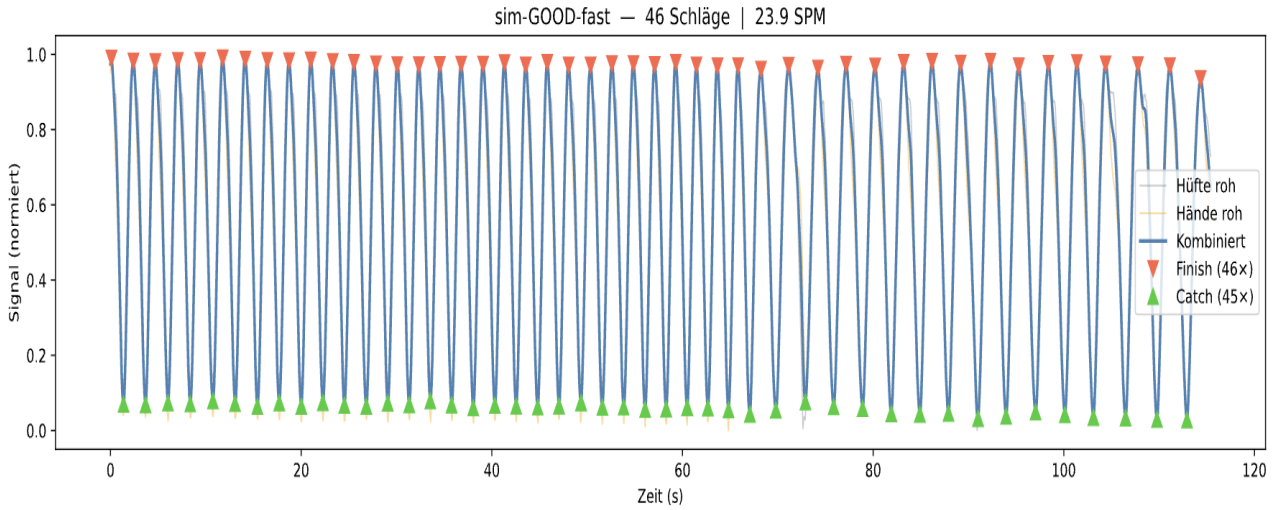
PIPELINE

Segmenting strokes

Goal. Count strokes and locate the motion phases — catch and finish.

How. Peak detection on the smoothed motion signal splits the sequence into individual strokes of ~24–28 frames each.

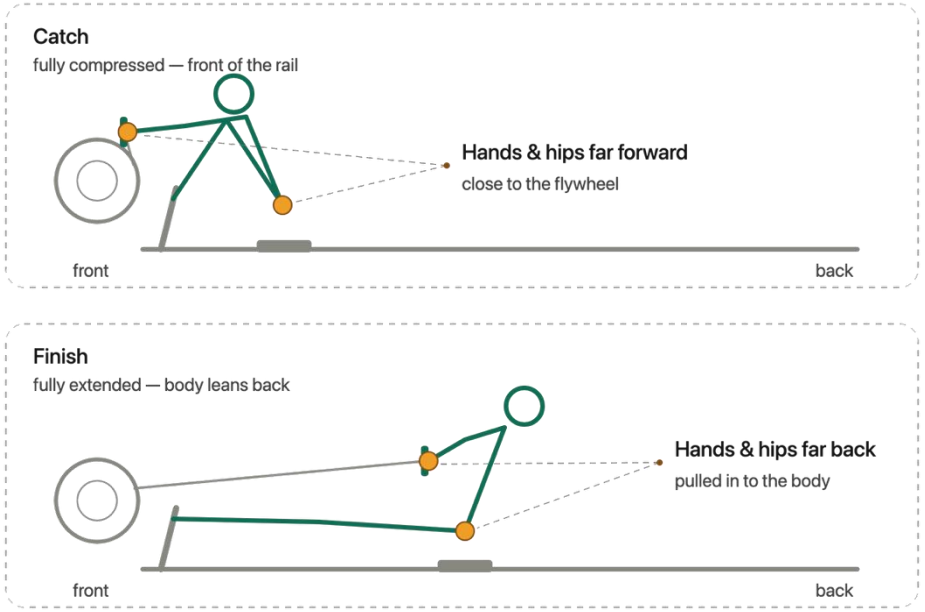
Detected strokes over time



Each peak marks one stroke; phase boundaries give catch and finish.

PIPELINE

Segmenting strokes



Segmentation · peak detection

```python
# ── Schritt 6: Zyklusdetektion ──────────────────────────────
hip_x_raw   = (df["left_hip_x"].values   + df["right_hip_x"].values)   / 2
wrist_x_raw = (df["left_wrist_x"].values + df["right_wrist_x"].values) / 2
window      = max(int(fps * 0.5) | 1, 5)
hip_smooth   = savgol_filter(hip_x_raw,   window_length=window, polyorder=3)
wrist_smooth = savgol_filter(wrist_x_raw, window_length=window, polyorder=3)

def norm01(x):
    return (x - x.min()) / (x.max() - x.min())

combined = (norm01(hip_smooth) + norm01(wrist_smooth)) / 2
min_dist = int(fps * 0.8)
endzug_frames,  _ = find_peaks( combined, distance=min_dist, prominence=0.01)
auslage_frames, _ = find_peaks(-combined, distance=min_dist, prominence=0.01)

n_strokes  = len(endzug_frames)
duration_s = len(hip_x_raw) / fps
rate       = n_strokes / duration_s * 60
msg = f"Schläge: {n_strokes} | Dauer: {duration_s:.1f}s | Rate: {rate:.1f} SPM"
print(msg); video_stats.append(msg)
```

METHODOLOGY

Stroke-aware train / test split

Random split confuses data

A stroke is ~24–28 consecutive, highly correlated frames. A plain shuffle-and-split puts neighbouring frames in both train and test.

Split by stroke, not by frame

Give every frame a `stroke_id`, then split on that id. It is reconstructed from the known frame and stroke counts per video.

How frames group into strokes

each block = one stroke (≈24–28 frames, shown reduced)

TRAIN

whole strokes only

TEST

held-out strokes

BASELINE · IMPLEMENTATION

Two baselines to beat

Dummy classifier

Majority baseline — always predicts the dominant GOOD class.

Random Forest

Class-weighted (balanced) so the rare BAD strokes actually count during training.

Implementation

```
# --- DummyClassifier (always predicts majority class = GOOD) ---
dummy = DummyClassifier(strategy="most_frequent", random_state=RANDOM_STATE)
dummy.fit(X_train, y_train)

# --- Random Forest with balanced class weights ---
rf = RandomForestClassifier(
    n_estimators=100,
    class_weight="balanced",
    random_state=RANDOM_STATE,
    n_jobs=-1
)
rf.fit(X_train, y_train)

print("Both models trained.")
```

BASELINE · RESULTS

Accuracy is misleading

Dummy (majority)

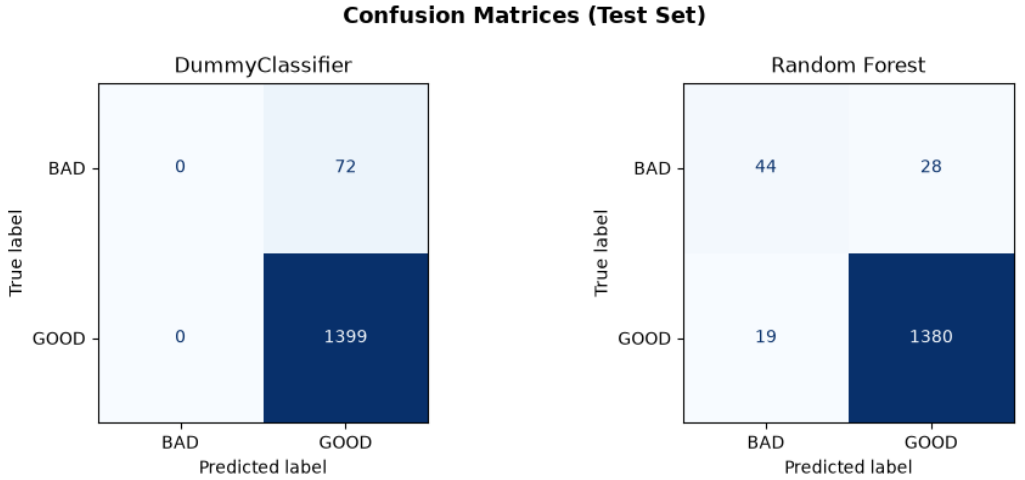
95.1%	0.50	0.49
Accuracy	ROC-AUC	Macro F1

Predicts GOOD for everything — BAD recall 0.00

Random Forest

96.8%	0.97	0.82
Accuracy	ROC-AUC	Macro F1

Actually separates the classes



Both models score ~96% accuracy, but only the Random Forest moves ROC-AUC to 0.97 and Macro F1 to 0.82.

THE CHALLENGE

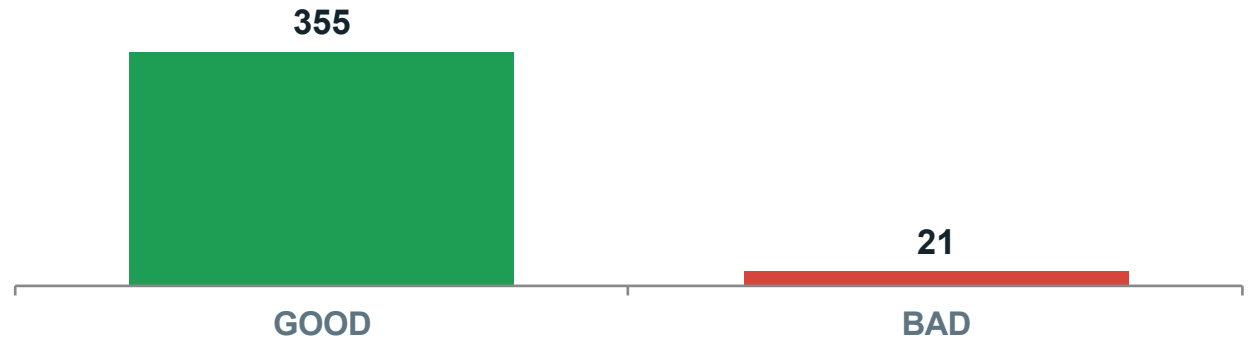
Biggest Fail: Imbalanced Dataset

355

good strokes

21

bad strokes



The Problem

95% accuracy is reachable by a model that **always predicts GOOD** — while learning nothing about technique.

LIMITATIONS

Main Concern: Data Quality

Only five videos

The original dataset is very small.

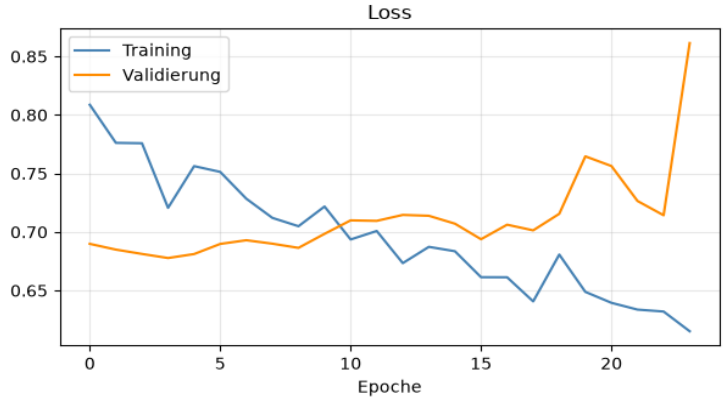
Bad strokes from only one source

Every low-quality stroke came from a single video.

Memorization risk

The model learns that one video instead of technique.

Evidence: Training a 3D-CNN



The 3D-CNN's training documentation confirms these concerns — strong train/validation divergence points to memorization.

LIMITATIONS

Main Concern: Data Quality

Only five videos

The original dataset is very small.

Bad strokes from one source

Every low-quality stroke came from a single video.

Memorization risk

The model may learn that one video instead of technique.

My solution

Steadily extend the dataset — record more sessions, leaving out bad strokes to stabilise the GOOD class.

Result so far

7 clean videos · **405** strokes · **0** frames without a pose

CURRENT DATASET

8 self-recorded videos

Video	Duration (s)	Orig. fps	Red. fps	Orig. frames	Red. frames	No-pose	Strokes	SPM
cla-BAD	50.6	30	10	1518	506	0	21	24.9
cla-GOOD-fast	209.3	30	10	6280	2094	0	88	25.2
cla-GOOD-slow	189.9	30	10	5698	1900	0	69	21.8
mar-GOOD-fast	140.6	30	10	4217	1406	0	62	26.5
mar-GOOD-slow	135.9	30	10	4076	1359	0	51	22.5
sim-GOOD-fast	115.4	30	10	3461	1154	0	46	23.9
sim-GOOD-slow	247.3	30	10	7419	2473	0	77	18.7
sim-GOOD-slow2	35.1	30	10	1054	352	0	12	20.5
Total	**1124.1**	**30**	**10**	**33 723**	**11 244**	**0**	**426**	**22.7**

8
videos

≈18.7 min
total

426
strokes

11 244
reduced frames

0
no-pose frames

NEW APPROACH

LSTM Autoencoder

Anomaly detection. Trained only on good strokes, the model reconstructs correct technique well — faulty strokes reconstruct poorly and stand out by a high error.

Autoencoding

Train on good strokes only — learn to reconstruct correct technique.

Sliding windows

Process sequences, not single frames (window 24, stride 12).

Threshold

Set from the training MSE; strokes whose error exceeds it are flagged BAD.

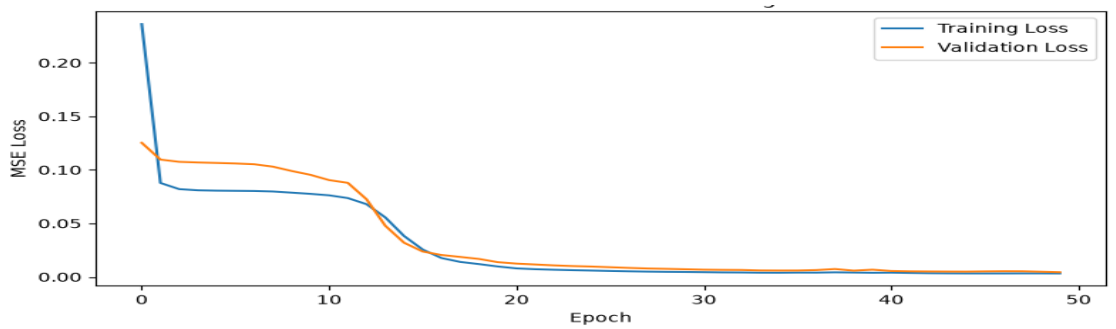
LSTM Autoencoder · Implementation

```
n_timesteps = WINDOW_SIZE
n_features  = X_train.shape[2]
LATENT_DIM  = 32

def build_lstm_autoencoder(timesteps, features, latent_dim):
    inputs = keras.Input(shape=(timesteps, features), name='input')

    # Encoder
    x = layers.LSTM(128, return_sequences=False, name='encoder_lstm')(inputs)
    bottleneck = layers.Dense(latent_dim, activation='relu', name='bottleneck')(x)

    # Decoder
    x = layers.RepeatVector(timesteps, name='repeat')(bottleneck)
    x = layers.LSTM(128, return_sequences=True, name='decoder_lstm')(x)
    outputs = layers.TimeDistributed(layers.Dense(features), name='output')(x)

    return keras.Model(inputs, outputs, name='LSTM_Autoencoder')

model = build_lstm_autoencoder(n_timesteps, n_features, LATENT_DIM)
model.compile(optimizer='adam', loss='mse')
model.summary()
```

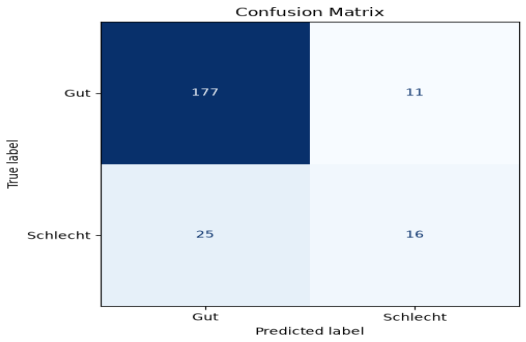
NEW APPROACH · RESULTS

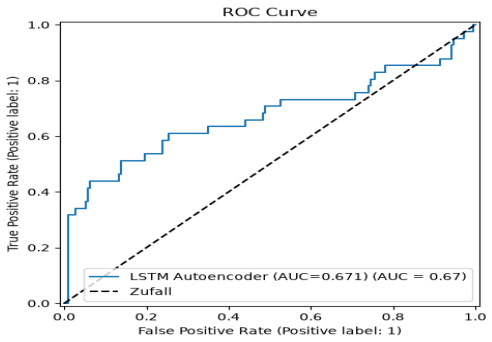
First autoencoder results

Training & validation loss



Confusion matrix & ROC curve





0.84	0.67	0.69
Accuracy	ROC-AUC	Macro F1

GOOD strokes

0.91	0.88	0.94
F1	Precision	Recall

BAD strokes

0.47	0.59	0.39
F1	Precision	Recall

Good strokes are caught well; bad strokes stay hard (recall 0.39) — more varied bad examples are needed.

OUTLOOK

Where this goes next

Done so far

- End-to-end pipeline: MediaPipe → features → model
- Normalization across angles & body proportions
- Stroke segmentation and leak-free, stroke-aware split
- Supervised baseline — Random Forest at Macro F1 0.82
- LSTM autoencoder built and evaluated (ROC-AUC 0.67)
- Dataset extended to 8 videos incl. first bad-technique clip

Next steps

- Record more & more varied BAD strokes (rowers, error types)
- Lift bad-stroke recall beyond 0.39 — tune threshold & window
- Benchmark the autoencoder against the RF baseline

One Day:

- Per-phase feedback on catch and finish

Thank you · questions welcome